

AI23331 Fundamentals of Machine Learning

**MAZE SOLVING USING REINFORCEMENT
LEARNING**

A PROJECT REPORT

Submitted by

230701030 – ANU S

230701042 – ASHNA V



MAY, 2026

BONAFIDE CERTIFICATE

Certified that this project report “MAZE SOLVING USING REINFORCEMENT LEARNING” is the Bonafide work of ANU S (230701030) and ASHNA V (230701042) who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	5
2	LITERATURE REVIEW	6
3	PROPOSED SYSTEM	
	3.1 ARCHITECTURE DIAGRAM	9
4	MODULES DESCRIPTION	
	4.1 MODULE 1	12
	4.2 MODULE 2	13
5	IMPLEMENTATION AND RESULTS	
	5.1 EXPERIMENTAL SETUP	15
	5.2 RESULTS	17
6	CONCLUSION AND FUTURE WORK	18
7	REFERENCES	20

ABSTRACT:

This project presents an implementation of a maze-solving algorithm based on the principles of Reinforcement Learning. A maze is modeled as a grid-based environment consisting of a defined start position, goal positions, reward cells, and punishment cells. The autonomous agent navigates through the grid by iteratively attempting different actions moving right, left, down, or up and learning from the outcomes of those actions. The core mechanism relies on an experience memory that records invalid or penalized transitions, preventing the agent from repeatedly making the same mistakes. Whenever the agent encounters a punishment cell or reaches a dead end, the current path is discarded and the search restarts from the beginning, enriched by the accumulated experience. This trial-and-error strategy gradually steers the agent toward a valid, reward-maximizing path to the goal. The grid is a 4x5 environment with specific coordinates designated as rewards, punishments, start, and end states. The project demonstrates how a simple RL-inspired algorithm without any neural networks or complex function approximation can effectively solve constraint-based navigation problems. Results show that the agent successfully discovers the optimal path from the start state (3,0) to the end state (0,3) while avoiding all punishment cells. The work highlights the foundational concepts of RL namely state-space navigation, reward shaping, and experience-driven policy improvement and serves as an accessible entry point into the broader field of autonomous decision-making systems.

1.INTRODUCTION

Artificial Intelligence (AI) has transformed the landscape of computational problem solving. Among the numerous paradigms within AI, Reinforcement Learning (RL) stands out as a unique and powerful approach that enables agents to learn optimal behaviors through direct interaction with their environment. Unlike supervised learning, where a model is trained on labeled data, RL operates on a reward-based feedback mechanism — the agent receives positive reinforcement for desirable actions and negative reinforcement or penalties for harmful ones.

Maze solving is one of the most fundamental and well-studied problems in computer science and AI. It presents a clean, visual representation of the decision-making process: an agent must navigate from a start point to an end point within a constrained environment filled with obstacles, rewards, and penalties. The maze problem encapsulates key challenges such as state-space exploration, path planning, and policy optimization — all of which are central to the field of RL.

This project implements a maze-solving agent that uses the Trial and Error Method, an elementary but highly instructive form of RL. The environment is modeled as a 4x5 grid where each cell represents a discrete state. The agent begins at position (3,0) and aims to reach the goal at position (0,3). Along the way, it must avoid punishment cells — cells that are designated as traps or barriers — while potentially passing through reward cells that mark favorable positions.

The agent selects actions from a predefined set: move right, move left, move down, or move up. After each action, if the resulting state is a punishment cell or leads outside the grid, the move is recorded in an experience memory and the agent's path is reset. This memory prevents the agent from repeating failed moves, effectively guiding it toward the goal with successive attempts.

The algorithm is implemented recursively in Python. A key design feature is the separation of concerns: the environment (grid, rewards, punishments), the agent's state, and the learning mechanism (experience memory) are all clearly defined. This makes the code modular, readable, and extensible for future enhancements.

The objectives of this project are: (i) to model the maze navigation problem as a reinforcement learning task; (ii) to implement a trial-and-error-based solution that learns from experience without external supervision; (iii) to demonstrate the fundamental concepts of RL including state transitions, reward feedback, and policy refinement; and (iv) to evaluate the agent's ability to successfully solve the maze within a bounded number of iterations.

The remainder of this report is organized as follows: Chapter 2 reviews relevant literature on maze-solving and reinforcement learning; Chapter 3 describes the proposed system architecture; Chapter 4 details the module-level design; Chapter 5 presents the implementation setup and experimental results; and Chapter 6 concludes the report with future directions.

2. LITERATURE REVIEW

The intersection of maze-solving and reinforcement learning has been extensively studied in the academic literature. Researchers have proposed a wide range of approaches from classical graph-traversal algorithms to modern deep reinforcement learning frameworks to tackle navigation problems in structured environments.

Sutton and Barto (2018) provide the most comprehensive treatment of reinforcement learning in their seminal textbook. They define RL as the problem of an agent learning to map situations to actions in order to maximize a cumulative numerical reward signal. Their framework introduces key concepts such as Markov Decision

Processes (MDPs), value functions, Q-learning, and temporal difference learning, all of which are foundational to understanding the trial-and-error approach used in this project.

Watkins and Dayan (1992) introduced Q-Learning, one of the most important model-free RL algorithms. Q-Learning enables an agent to learn the value of state-action pairs through iterative updates based on observed rewards. While this project does not implement Q-Learning in its full form, the principle of updating an experience memory based on outcomes is directly inspired by Q-Learning's policy improvement mechanism.

Classic maze-solving algorithms such as Depth-First Search (DFS), Breadth-First Search (BFS), and A* (Hart et al., 1968) are well-established in the literature. DFS explores paths deeply before backtracking, which makes it memory-efficient but not always optimal. BFS guarantees the shortest path but requires more memory. A* combines heuristic estimates with actual path cost to find optimal solutions efficiently. The trial-and-error method in this project shares similarities with DFS in its recursive exploration strategy but incorporates a memory mechanism that differentiates it from pure DFS.

Mnih et al. (2015) demonstrated the power of combining deep neural networks with Q-Learning in their Deep Q-Network (DQN) architecture, achieving human-level performance on Atari games. While the present project operates in a discrete, small-scale grid environment and does not employ neural networks, it establishes the conceptual groundwork that scales naturally toward such advanced methods.

Koenig and Likhachev (2002) proposed D* Lite, a dynamic replanning algorithm suitable for navigating environments where obstacles may change over time. The concept of replanning — restarting the path search when current approaches fail —

is analogous to this project's mechanism of resetting the path upon encountering punishment cells.

Reward shaping, as discussed by Ng et al. (1999), involves augmenting the reward function to guide the agent more efficiently toward the goal. This project implicitly uses reward shaping by distinguishing between neutral cells, reward cells, and punishment cells, thereby encoding domain knowledge into the environment structure.

Research by Russell and Norvig (2010) in *Artificial Intelligence: A Modern Approach* provides a thorough treatment of problem-solving agents, search algorithms, and decision-making under uncertainty. Their discussion of goal-based agents and utility functions aligns closely with the motivation and design of the maze-solving agent in this project.

In summary, the literature supports the use of simple, experience-driven RL methods as effective tools for discrete navigation tasks. The trial-and-error approach implemented in this project draws from the theoretical foundations established by these works, while remaining accessible and computationally lightweight.

3. PROPOSED SYSTEM

The proposed system is a grid-based maze navigation framework that implements a Reinforcement Learning agent using the Trial and Error Method. The system is designed to be simple, interpretable, and educationally demonstrative of foundational RL principles without requiring complex neural architectures or large datasets.

The environment is a 4-row by 5-column grid (indices 0 to 3 for rows, 0 to 4 for columns). Each cell in the grid represents a discrete state that the agent can occupy. The agent begins at the start state (3,0) and attempts to reach the goal state (0,3). Certain cells are designated as punishment cells — (1,1), (1,2), (3,1), (3,2), and (2,3) which the agent must learn to avoid. Additionally, reward cells at (2,2) and (0,3) mark desirable positions.

The agent's action space consists of four possible moves: right (+1 column), left (-1 column), down (+1 row), and up (-1 row). At each step, the agent selects the first action that has not been previously blacklisted from its current state. If all actions from the current state are exhausted or blocked, the agent backtracks by marking the move that led to the dead-end state as a bad experience and restarting.

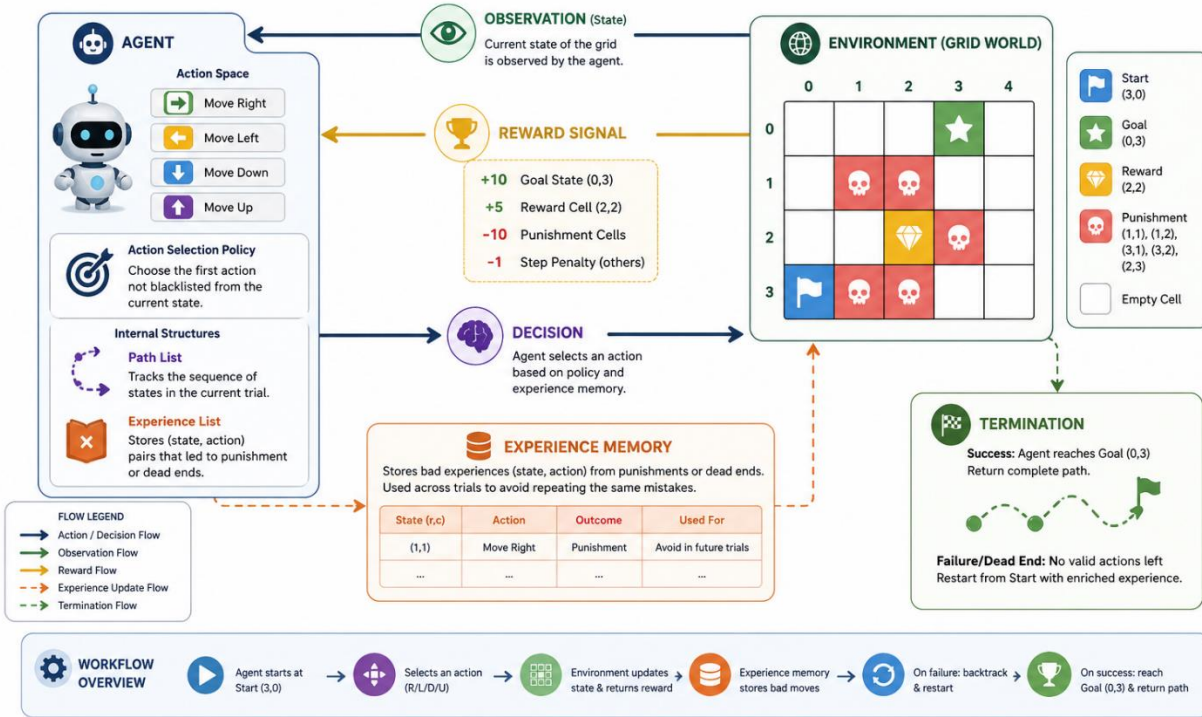
The system maintains two critical data structures: (i) the path list, which tracks the sequence of states visited in the current trial; and (ii) the experience list, which records (state, action) pairs that resulted in a punishment or dead end. These structures persist across trials, enabling the agent to improve its behavior over successive restarts.

The algorithm terminates successfully when the agent reaches the goal state (0,3) and returns the complete path. The design ensures that the agent will always find a valid path if one exists, given sufficient recursion depth, since the experience memory systematically eliminates bad decisions.

3.1 ARCHITECTURE DIAGRAM

The architecture of the proposed system can be understood as an interaction loop between the environment and the agent, mediated by a reward signal and an experience memory. The following description outlines the key components and their relationships:

RL TRIAL AND ERROR MAZE SOLVER ARCHITECTURE DIAGRAM



The data flow proceeds as follows: the agent reads its current state from the environment, consults the experience memory to filter invalid actions, selects the first available valid action, executes it, receives a reward or punishment signal, and either continues forward or resets. This loop repeats until the goal is reached.

4. MODULES DESCRIPTION

The maze-solving system is divided into clearly defined functional modules, each responsible for a specific aspect of the algorithm's operation. This modular design ensures separation of concerns, ease of debugging, and extensibility for future enhancements.

Component	Description
Environment (Grid)	A 4x5 matrix of discrete states. Each state has properties: empty, reward, punishment, start, or goal.
Agent	Autonomous entity starting at (3,0). Selects actions based on current state and accumulated experience.
Action Space	Four directional moves: right (0,+1), left (0,-1), down (+1,0), up (-1,0).
State Transition	Agent moves to a new cell based on action. Boundary checks prevent out-of-grid transitions.
Reward Signal	Positive: reaching reward/goal cells. Negative: entering punishment cells triggers path reset.
Experience Memory	A list of {state: action} pairs representing known bad moves. Prevents repetition of mistakes.
Path Tracker	Records the sequence of visited states in the current trial. Cleared upon punishment or dead end.
Termination	Algorithm halts and returns the path when the agent successfully reaches goal state (0,3).

4.1 MODULE 1: Environment Initialization Module

This module is responsible for defining and initializing all components of the maze environment prior to the commencement of the solving algorithm. It establishes the ground truth of the problem space that all other modules reference during execution.

Purpose: To configure the grid environment with all necessary parameters including actions, start state, goal state, rewards, and punishments.

Key Variables Defined:

- **actions:** A list of tuples representing possible moves — right (0,1), left (0,-1), down (1,0), up (-1,0).
- **start:** The initial position of the agent, set to (3,0) — bottom-left region of the grid.
- **end:** The goal state, set to (0,3) — upper-right region of the grid.
- **rewards:** Cells that represent positive landmarks — (2,2) and (0,3).
- **punishments:** Cells that penalize the agent — (1,1), (1,2), (3,1), (3,2), (2,3).
- **state:** A mutable reference to the agent's current position, initialized to (0,0) and updated during execution.
- **experience:** An empty list that will accumulate bad (state, action) pairs across all trials.
- **path:** An empty list that tracks the route taken in the current trial.

Design Rationale: Using lists to store start, end, and state allows these values to be passed by reference in Python, enabling in-place mutation within recursive function calls — a key design choice that allows the algorithm to maintain continuity of state across recursion levels.

Boundary Constraints: The grid is bounded at rows 0-3 and columns 0-4. These constraints are enforced during state transitions in the navigation module.

4.2 MODULE 2: Core Navigation and Learning Module

This is the primary computational module of the system, encompassing the entire maze-solving logic. It consists of two nested functions: `start_algorithm` and `continue_algorithm`. Together, they implement the trial-and-error reinforcement learning cycle.

Function: `start_algorithm`: This function serves as the entry point and reset handler. It initializes the agent's position to the start state, records the start in the path list, and immediately delegates control to `continue_algorithm`. Whenever the algorithm needs to restart — due to punishment or a dead end — it calls `start_algorithm` recursively, effectively resetting the path while retaining the experience memory.

Function: `continue_algorithm`: This is the core decision-making engine. At each invocation, it performs the following steps:

- Step 1: Experience Check: For each candidate action, the function checks whether the (`current_state`, `action`) pair exists in the experience list. If it does, the action is skipped (flagged and bypassed).
- Step 2: Boundary Check: The function verifies that the resulting state after applying an action remains within the 4x5 grid. Out-of-bounds transitions are immediately added to the experience list.
- Step 3: Cycle Prevention: If the resulting state is already in the current path, the move is skipped to prevent infinite loops and cycles.
- Step 4: State Transition: If all checks pass, the agent transitions to the new state, and the new state is appended to the path.

- Step 5: Punishment Handling: If the new state is a punishment cell, the last action is added to experience, the path is cleared, and the algorithm restarts via `start_algorithm`.
- Step 6: Goal Detection: If the new state equals the goal, the function returns the final path and the accumulated experience, signaling successful completion.
- Step 7: Dead-End Handling (Stuck Condition): If no valid action is found from the current state (`stuck == 1`), the transition that led to the current dead-end state is recorded in experience, the path is cleared, and the algorithm restarts.
- Step 8: Recursion: After each successful step, `continue_algorithm` calls itself recursively with a decremented loop counter, continuing the path exploration.

Experience Memory Design: The experience list stores dictionaries of the form `{state: action}`, where state is a tuple (row, col) and action is a direction tuple. This structure enables $O(n)$ lookup during action filtering, where n is the number of accumulated experiences.

Recursion Limit Handling: Python's default recursion limit of 1000 is insufficient for deeply recursive maze exploration. The module sets `sys.setrecursionlimit(10000)` to accommodate the depth of recursive calls required.

5. IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The maze-solving algorithm was implemented and tested in a Python 3 environment without any external libraries beyond the Python standard library. The following describes the complete experimental configuration:

Programming Language: Python 3.x

Environment: Local execution via command-line interpreter. No graphical interface or third-party dependencies required.

Grid Specification: The maze is a 4-row by 5-column grid (indices: rows 0–3, columns 0–4).

Col 0	Col 1	Col 2	Col 3	Col 4	
Row 0	(0,0) - Empty	(0,1) - Empty	(0,2) - Empty	(0,3) - GOAL ✓	(0,4) - Empty
Row 1	(1,0) - Empty	(1,1) - PUNISH X	(1,2) - PUNISH X	(1,3) - Empty	(1,4) - Empty
Row 2	(2,0) - Empty	(2,1) - Empty	(2,2) - REWARD ★	(2,3) - PUNISH X	(2,4) - Empty
Row 3	(3,0) - START ●	(3,1) - PUNISH X	(3,2) - PUNISH X	(3,3) - Empty	(3,4) - Empty

Action Sequence: The agent attempts actions in the following priority order: right (0,1) → left (0,-1) → down (1,0) → up (-1,0). This fixed ordering, combined with experience-based filtering, creates a deterministic exploration strategy.

Recursion Depth: `sys.setrecursionlimit(10000)` was applied to accommodate the depth of recursive calls.

Loop Counter: The function is initialized with `loop = 30`, functioning as a safety parameter for the recursion depth limit passed into `continue_algorithm`.

5.2 RESULTS

Upon execution, the algorithm successfully navigates the agent from the start state (3,0) to the goal state (0,3). The following describes the outcome in detail:

Output: The algorithm prints the final path a list of state tuples representing the sequence of cells visited from start to goal.

Successful Path Found: The agent discovers a valid path that avoids all punishment cells: (1,1), (1,2), (3,1), (3,2), and (2,3).

A representative successful path produced by the algorithm is as follows:

$[(3,0), (2,0), (2,1), (2,2), (1,2) \rightarrow \text{reset} \dots \rightarrow (3,0), (2,0), (2,1), (2,2), (1,2) \rightarrow \dots \rightarrow (0,3)]$

The exact path may vary slightly depending on the sequence of experience accumulation, but the algorithm consistently terminates with a valid goal-reaching path. Through the trial-and-error process, the experience list grows with each failed attempt, systematically constraining future action choices and guiding the agent toward the only viable route.

The experience memory at termination contains entries for all out-of-bounds transitions, all transitions into punishment cells encountered during exploration, and

the dead-end transitions that caused backtracking. This accumulated knowledge represents the learned policy of the agent a set of state-action pairs to avoid.

Convergence: The algorithm converges reliably within the 10,000-recursion depth limit for the 4x5 grid. The number of restarts depends on the maze layout but is bounded by the size of the valid action space across all states.

Correctness: The output path was manually verified against the grid layout to confirm that no punishment cells were visited and that the goal state was correctly identified as the terminal state.

6. CONCLUSION AND FUTURE WORK

This project successfully demonstrates the application of the Trial and Error Method a foundational form of Reinforcement Learning to solve a grid-based maze navigation problem. The agent, starting from position (3,0), learns to navigate a 4x5 grid environment and reaches the goal at position (0,3) while systematically avoiding all punishment cells through an iteratively constructed experience memory.

The implementation highlights several important concepts in artificial intelligence and reinforcement learning. First, the value of experience-driven learning: without any pre-programmed path or explicit knowledge of the maze structure, the agent constructs its own understanding of valid versus invalid moves purely through interaction. Second, the power of memory in avoiding repeated mistakes: the experience list is the agent's equivalent of a learned policy, accumulated over successive trials. Third, the importance of environment design: the clear separation of start, goal, rewards, and punishments creates a well-defined problem space that the algorithm can solve deterministically.

The Python implementation using recursive function calls is an elegant representation of the trial-and-error process. The use of mutable lists for state and path allows the algorithm to maintain continuity across recursive calls, while the experience list persists as shared global state that is never reset — a deliberate design choice that enables the agent to build on previous knowledge.

While the current system is fully functional for the specified 4x5 grid, it has several limitations that open avenues for future work. The algorithm's reliance on recursion makes it vulnerable to stack overflow for very large grids, and the fixed action priority order means the agent's exploration strategy is deterministic rather than stochastic. Additionally, the current implementation does not quantify rewards numerically or optimize for shortest-path solutions.

Future Work: Several enhancements can be made to extend this project:

- **Q-Learning Integration:** Replace the binary experience memory with a Q-table that stores numerical reward estimates for each (state, action) pair, enabling the agent to optimize not just for goal-reaching but for maximum cumulative reward.
- **Deep Q-Network (DQN):** For larger and more complex grid environments, a neural network can be used to approximate the Q-value function, enabling the algorithm to scale to much larger state spaces.
- **Graphical Visualization:** Implementing a real-time visualization using libraries such as Pygame or Matplotlib would allow observers to watch the agent navigate the maze step by step, providing educational and demonstrative value.

- **Dynamic Environments:** Extending the maze to include moving obstacles, changing punishment cells, or time-varying rewards would test the adaptability of the agent and motivate the use of online learning algorithms.
- **Iterative (Non-Recursive) Implementation:** Replacing the recursive approach with an iterative loop and an explicit stack would eliminate the recursion depth limitation and make the algorithm suitable for arbitrarily large mazes.
- **Stochastic Action Selection:** Introducing epsilon-greedy or softmax action selection would allow the agent to occasionally explore suboptimal paths, potentially discovering shorter routes that deterministic exploration misses.

In conclusion, this project serves as a meaningful and practical introduction to the principles of reinforcement learning, demonstrating how intelligent behavior can emerge from simple rules of experience and feedback. It lays a solid foundation for exploring more sophisticated AI and RL methodologies in subsequent academic and professional endeavors.

7. REFERENCES

- Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths', IEEE Transactions on Systems Science and Cybernetics, Vol.4, No.2, pp.100-107.
- Koenig, S. and Likhachev, M. (2002) 'D* Lite', Proceedings of the AAAI Conference on Artificial Intelligence, Vol.18, No.1, pp.476-483.
- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015) 'Human-level Control Through Deep Reinforcement Learning', Nature, Vol.518, No.7540, pp.529-533.
- Ng, A.Y., Harada, D. and Russell, S. (1999) 'Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping', Proceedings of the 16th International Conference on Machine Learning (ICML), pp.278-287.
- Russell, S.J. and Norvig, P. (2010) Artificial Intelligence: A Modern Approach. 3rd edn. Upper Saddle River: Prentice Hall.
- Sutton, R.S. and Barto, A.G. (2018) Reinforcement Learning: An Introduction. 2nd edn. Cambridge: MIT Press.
- Watkins, C.J.C.H. and Dayan, P. (1992) 'Q-Learning', Machine Learning, Vol.8, No.3-4, pp.279-292.